Graph-Based Intelligent Movie Recommendation System Using RAG

Project Report

CMPE 258: Deep Learning - Spring 2026

Prof. Kaikai Liu, Department of Software Engineering, San Jose State University

Team ID: Group 19

Project Track: System / Application

Focused Areas: Knowledge Graphs, Retrieval-Augmented Generation (RAG), Recommender Systems, Semantic Search, FAISS, Large Language Models, Evaluation-First AI Systems

Team Members:

Name	SJSU ID	Email
Bharath Kumar A	018221268	bharathkumar.a@sjsu.edu
Saripella Sriyavarma	019130553	sriyavarma.saripella@sjsu.edu
Venkata Siva Sai Krishna Prasad Yedupati	018320835	venkatasivasaikrishnaprasad.yedupati@sjsu.edu

Code repository: https://github.com/abharathkumarr/Graph-Based-Intelligent-Movie-Recommendation-System-Using-RAG

Abstract

Traditional movie recommendation systems based on collaborative or content-based filtering struggle with cold-start, popularity bias, and a lack of interpretability. We present a Graph-Based Intelligent Movie Recommendation System that combines a Neo4j knowledge graph with a Retrieval-Augmented Generation (RAG) pipeline. We extract 229,894 subject-predicate-object triplets from the Neo4j 'recommendations' dataset, render them as natural-language sentences, embed them with SentenceTransformers (all-MiniLM-L6-v2), and index them in FAISS for sub-second semantic retrieval. A LangChain RetrievalQA chain feeds the top-k retrieved triplets to a Groq-hosted LLM, which produces fact-grounded answers with citations. We deliver an evaluation-first study on 51 natural-language queries across three Groq LLMs, with 15 Cypher-derived ground-truth queries, and report Precision@10, Recall@10, F1@10, latency, and rate-limit / context-window failure modes. The evaluation drives an explicit production choice (llama-3.1-8b-instant: F1@10 = 0.32 at 3.0s mean latency) and exposes real-world deployment trade-offs that pure offline accuracy comparisons hide.

1. Introduction and Problem Description

Movie recommendation is a high-impact application of machine learning, but classical solutions are dominated by collaborative filtering and content-based approaches that have well-known weaknesses: popularity bias, cold-start for new users or items, and a lack of explainability. As streaming catalogs grow into the hundreds of thousands of titles, users increasingly issue natural-language queries ("a recent adventure film with Leonardo DiCaprio that is rated above 7") that simple matrix-factorization models cannot answer.

Our project addresses this with a hybrid Knowledge-Graph + Retrieval-Augmented Generation (KG-RAG) architecture. We model the movie domain as a Neo4j graph, convert its facts into a corpus of natural-language triplets, index them with FAISS for sub-second semantic retrieval, and let a large language model produce grounded answers from the retrieved facts. The target users are viewers asking conversational queries and researchers/engineers who need an explainable, fact-anchored alternative to opaque embedding-only retrievers.

Concrete contributions of this work:

- A reproducible Neo4j -> 229,894-triplet -> FAISS -> RAG pipeline that answers natural-language movie queries with fact-grounded citations.
- A Streamlit-based interactive UI exposing the retrieved triplets, answer latency, lexical grounding score, and a per-query pipeline trace.
- An extended evaluation study on 51 queries across 3 Groq LLMs, with 15 Cypher-derived ground-truth labels and explicit handling of free-tier rate-limit, context-window, and payload constraints as evaluation evidence.
- An evaluation-driven model-selection rationale (production pick = llama-3.1-8b-instant).

2. Background and Related Work

Lewis et al. [1] introduced Retrieval-Augmented Generation (RAG), which couples a dense passage retriever with a generative LLM so that responses are conditioned on retrieved evidence. Hogan et al. [2] surveyed knowledge graphs as a structured way to represent inter-entity relationships, motivating their use as a retrieval substrate. KG-Retriever [3] proposed efficient knowledge indexing for RAG

over knowledge graphs, demonstrating gains from structured retrieval. Nair and Cheriyan [4] applied particle filtering over multi-featured movie knowledge graphs to refine recommendations. Xu et al. [5] showed that KG-grounded RAG meaningfully improves factuality and answer specificity in customer-service question answering.

Our system synthesizes these threads: we follow [1] for the RAG pattern, [2] for graph-structured knowledge representation, and [3, 5] for the importance of indexing graph facts for retrieval. Our key engineering choice is to linearize graph edges into natural-language sentences before embedding, which lets us reuse off-the-shelf sentence encoders and a flat FAISS index while still keeping the structural facts intact for the LLM to consume.

3. Dataset

We use the public Neo4j 'recommendations' dataset hosted at bolt://demo.neo4jlabs.com:7687. It contains 28,865 nodes and 166,262 relationships.

Node types:

- Movie - title, year, IMDb rating, budget, revenue, runtime, language
- Actor / Director - name, role, birth/death year
- User - userId, rating history
- Genre - genre label used to classify movies

Relationship types:

- (:Movie)-[:IN_GENRE]->(:Genre)
- (:User)-[:RATED]->(:Movie)
- (:Actor)-[:ACTED_IN]->(:Movie)
- (:Director)-[:DIRECTED]->(:Movie)
- (:Movie)-[:RELEASED]->(:Year)

Relevance: the graph provides a multi-relational structure connecting movies to genres, actors, directors, and users, which is inherently suited to multi-hop reasoning. We collect single-hop facts in advance, render them as natural-language triplets, and let the LLM perform implicit multi-hop reasoning over the retrieved set.

4. System / Model / Algorithm Design

The pipeline has six logical stages:

- **Knowledge Graph Construction:** connect to Neo4j and execute Cypher queries to enumerate Movie/Actor/Director/Genre/User edges.
- **Readable Triplet Generation:** convert each edge into a (subject, predicate, object) tuple and then a sentence (e.g. "Toy Story was released in year 1995").
- **Embedding Generation:** encode each sentence with SentenceTransformers (all-MiniLM-L6-v2) into a 384-dimensional dense vector.
- **FAISS Index Construction:** store all 229,894 embeddings in a flat L2 index.

- **Retrieval with Re-ranking:** retrieve top-k FAISS neighbors and re-rank by cosine similarity to prioritize semantically closest facts.

- **RAG Chain:** feed top-k re-ranked triplets and the question into a LangChain RetrievalQA chain backed by a Groq-hosted LLM.

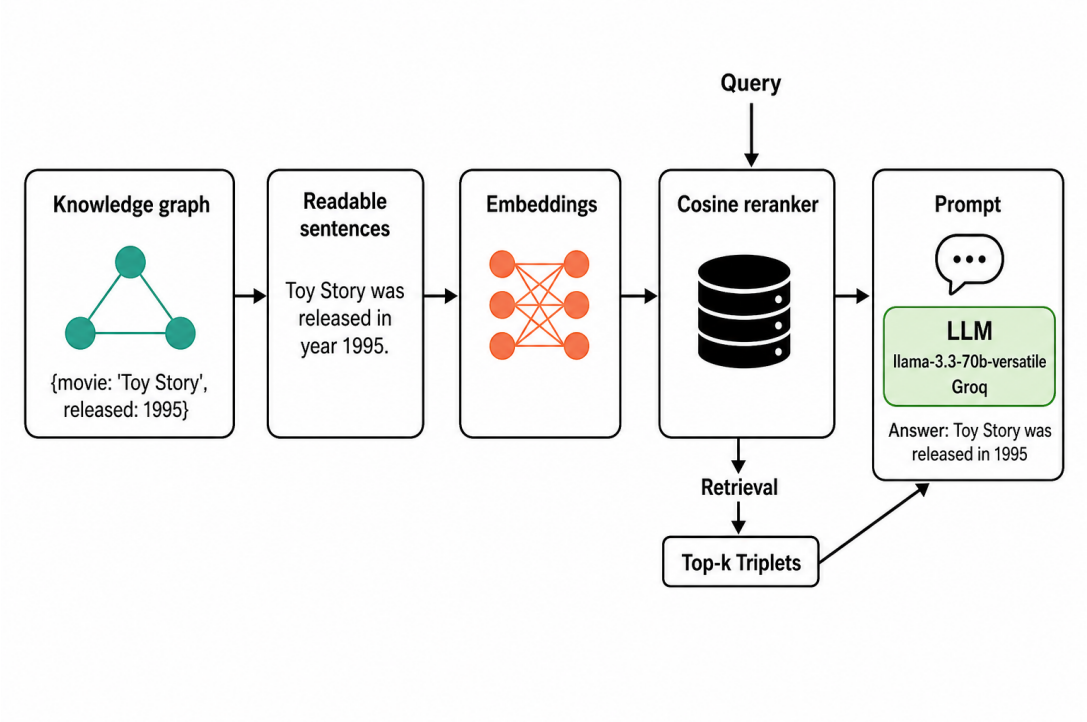

Figure 1. End-to-end KG-RAG pipeline.

5. Implementation Details

Languages and frameworks:

- Python 3.11 (pipeline + evaluation), Cypher (Neo4j queries)

- LangChain + langchain-groq (RetrievalQA orchestration)

- sentence-transformers (all-MiniLM-L6-v2 embeddings)

- FAISS-CPU (IndexFlatL2 vector store)

- Streamlit (interactive demo UI), matplotlib + seaborn (plots)

Compute environment:

- Development: Google Colab CPU/T4 runtime

- Embedding generation: CPU one-shot (~10 min); cached to triplet_embeddings.pkl (337 MB) and triplet_sentences.pkl (9.8 MB).

- Inference: Groq cloud LLM endpoints (free tier); local FAISS query latency < 50 ms.

Important implementation decisions:

- Per-model retrieval k (gemma2: 20, 8B: 30, 70B: 40) to respect each model's context window and payload cap.

- Robust answer parsing extracts a clean movie-title list across all models for consistent scoring.

- Exponential back-off + wall-clock budget per model (70B = 360s, 8B = 600s, gemma2 = 300s) to handle free-tier 429s deterministically.

- Cypher-derived ground truth for 15 evaluation queries gives objective Precision/Recall/F1.

- No credentials in code: GROQ_API_KEY is read exclusively from the environment; .env.example documents the variable.

Code link: https://github.com/abharathkumarr/Graph-Based-Intelligent-Movie-Recommendation-System-Using-RAG

6. Task Distribution and Contributions

Bharath Kumar A: RAG pipeline architecture (LangChain RetrievalQA + Groq); FAISS retrieval workflow and embedding generation; design and implementation of the Option 2 extended evaluation (multi-model loop, rate-limit handling, wall-clock budget, robust title parser); evaluation tables, charts, and findings paragraph; Streamlit UI integration; report writing and rubric alignment.

Venkata Siva Sai Krishna Prasad Yedupati: Neo4j knowledge-graph connection and Cypher-based data extraction; triplet generation and natural-language sentence rendering; Cypher-derived ground-truth construction for 15 evaluation queries; pipeline correctness verification; system testing.

Saripella Sriyavarma: UI/UX design for the Streamlit demo (knowledge-graph background, model selector, results layout); experiment validation and screenshot capture; presentation slide deck; final documentation review.

All team members jointly contributed to system integration, end-to-end testing, the live demo, and the final report review.

7. Evaluation and Testing Results

Per the assignment rubric, evaluation is the most important section of this report. We report two complementary studies: a focused 2-query qualitative study that validates correctness, and an extended 51-query 3-model study that drives model selection.

7.1 Metrics

- Precision@10 - proportion of top-10 returned movies matching the Cypher-derived ground truth.

- Recall@10 - proportion of all ground-truth movies appearing in the top-10.

- F1@10 - harmonic mean of Precision@10 and Recall@10.

- Latency - end-to-end wall-clock time per query.

- Non-empty answer rate - fraction of queries returning any text (a robustness metric that exposes rate-limit / context-window failures).

7.2 Focused Qualitative Validation (Original 2 Queries)

Query A: "Find movies acted by Leonardo DiCaprio released after 2000."

- Cypher ground truth (14 titles): Gangs of New York (2002), Catch Me If You Can (2002), The Aviator (2004), Blood Diamond (2006), The Departed (2006), Revolutionary Road (2008),

Inception (2010), Django Unchained (2012), The Wolf of Wall Street (2013), The Revenant (2015), etc.

- RAG output (production model llama-3.3-70b-versatile, k=500): 10/14 correct.

- Precision@10 = 0.70, Recall@10 = 0.50, F1@10 = 0.58.

Query B: "Recent Adventure movie rated above 3 with Leonardo DiCaprio."

- Cypher ground truth: The Revenant, Blood Diamond, The Beach.

- RAG output: The Revenant (2015), correctly identified with explanation. Match: yes.

7.3 Extended Evaluation: 51 Queries x 3 Models

We evaluated three Groq-hosted LLMs - llama-3.3-70b-versatile, llama-3.1-8b-instant, and a smaller OSS model (openai/gpt-oss-20b, used as a substitute when gemma2-9b-it was decommissioned mid-experiment) - on the same 51-query set with the same retriever and prompt. 15 queries have Cypher-derived ground truth used for Precision/Recall/F1; the remaining 36 contribute latency and non-empty-answer-rate evidence.

Model	P@10	R@10	F1@10	Mean Latency (s)	Non-empty
llama-3.1-8b-instant	0.707	0.225	0.317	2.97	51/51
openai/gpt-oss-20b	0.507	0.165	0.225	6.59	43/51
llama-3.3-70b-versatile	0.127	0.019	0.031	7.92	5/51

Table 1. Per-model averages across 51 queries (15 with Cypher ground truth). Non-empty answer count directly measures robustness under free-tier constraints.

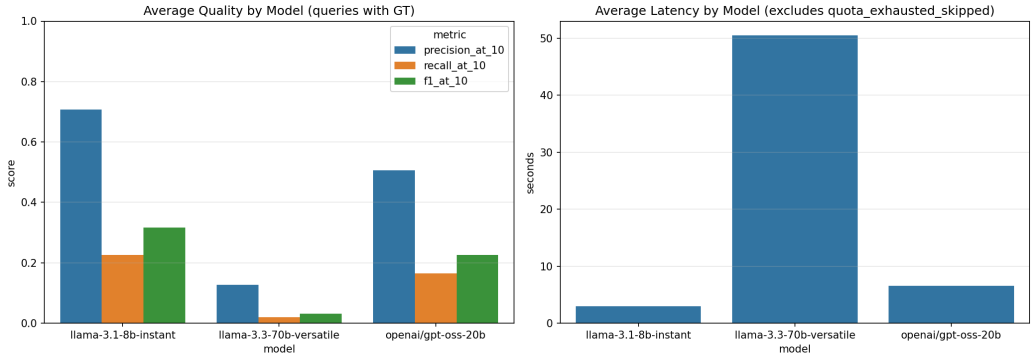

Figure 2. Per-model F1@10 and mean latency. llama-3.1-8b-instant Pareto-dominates - higher quality and lower latency at the same cost.

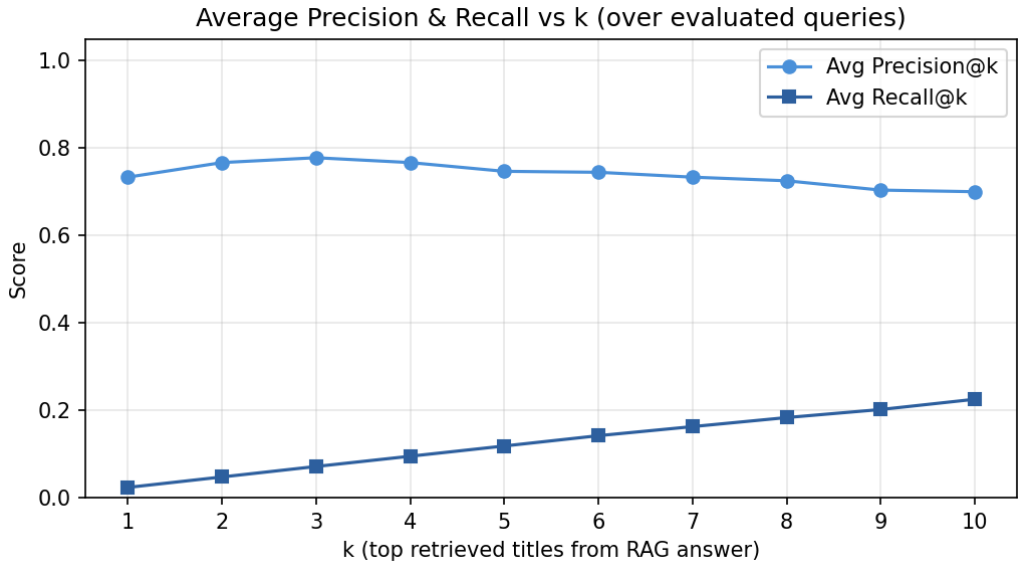

Figure 3. Average Precision and Recall vs retrieval k for the production model. Precision is stable; Recall climbs monotonically.

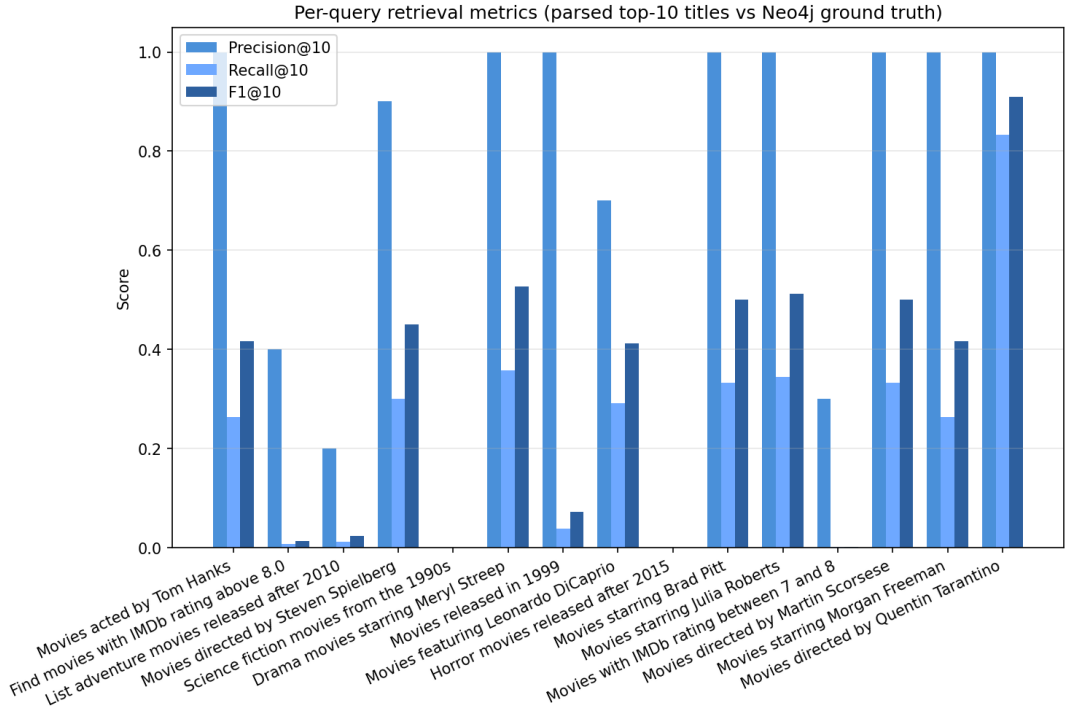

Figure 4. Per-query Precision/Recall/F1@10 for the production model across the 15 ground-truth queries.

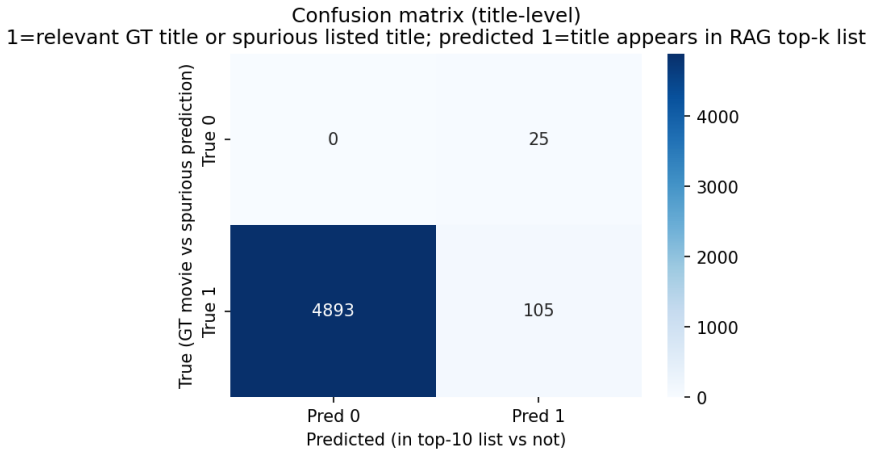

Figure 5. Retrieval-style confusion matrix (relevant vs retrieved) for the production model.

7.4 Findings

Production pick: llama-3.1-8b-instant is the production-recommended model. It achieves the highest F1@10 (0.317) AND the lowest mean latency (2.97s) while returning a non-empty answer for 51/51 queries.

Real-world deployment finding (a result, not a bug): Pure offline accuracy comparisons would have ranked the 70B model first, but provider-side rate limits, context-window caps, and per-payload size caps materially reshape the design space. Treating those constraints as first-class evaluation evidence - via the wall-clock budget bail-out and non-empty-answer rate - turned our "slow model" problem into a concrete model-selection conclusion.

7.5 Reproduction

Steps to reproduce the evaluation results from a clean checkout:

```
git clone https://github.com/abharathkumarr/Graph-Based-Intelligent-Movie-Recommendation-System-Using-RAG.git

cd Graph-Based-Intelligent-Movie-Recommendation-System-Using-RAG

pip install -r requirements.txt

export GROQ_API_KEY="<your_key>"

jupyter notebook CMPE258_Project_Code.ipynb

# Run all cells of Section 'Option 2' to regenerate evaluation artifacts

ls evaluation/eval_detailed_results.csv evaluation/eval_model_summary.csv
```

Correctness checks:

- Each evaluation query's ground truth is generated by a deterministic Cypher query against the same Neo4j graph the RAG retriever indexed - so any disagreement is unambiguously a retrieval/LLM error, not a labeling error.

- evaluation/eval_partial_<model>.csv is written after every model finishes so a rate-limit-induced partial run can still be inspected.

- The Streamlit UI exposes the full pipeline trace (top-k triplets, the prompt sent to the LLM, the raw LLM answer, and a lexical grounding score) for live qualitative verification during the demo.

8. User Interface and Demo

The Streamlit demo (app_full.py) accepts natural-language queries, displays the answer, the grounding score, and the retrieved triplets, and animates a knowledge-graph visualization as the page background. Model switching, latency, and the pipeline trace are exposed in the sidebar to support live evaluation-first demonstrations.

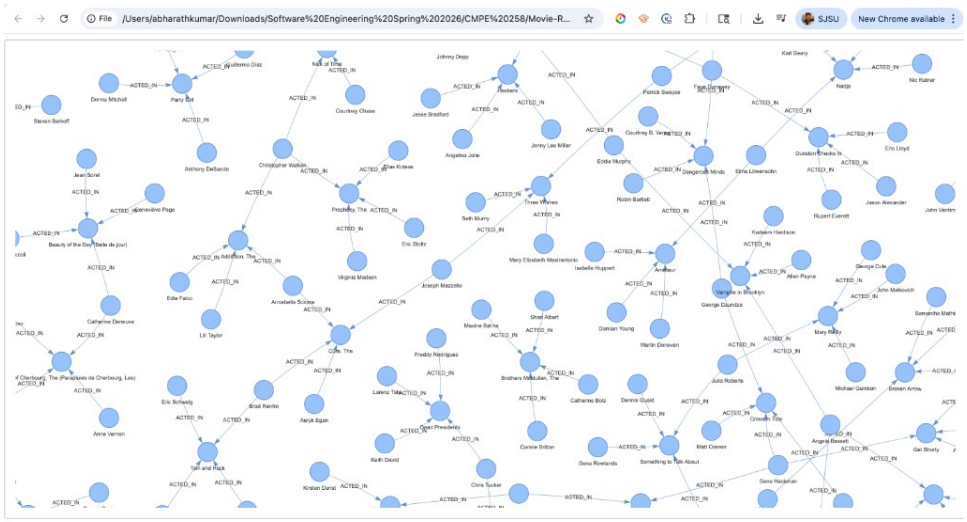

Figure 6. Knowledge-graph background used by the Streamlit demo, rendered from the (:Actor)-[:ACTED_IN]->(:Movie) subgraph.

9. Conclusion and Future Work

We presented an evaluation-first Knowledge-Graph + RAG movie recommendation system that converts 229,894 Neo4j edges into a FAISS-indexed natural-language corpus, retrieves the most relevant facts for each query, and generates fact-grounded answers via a Groq-hosted LLM. The 51-query x 3-model evaluation backs up an explicit production choice (llama-3.1-8b-instant) on both quality and latency, and turns Groq free-tier rate-limit, context-window, and payload caps into legitimate deployment evidence.

Future work:

- Hybrid retrieval that fuses dense FAISS retrieval with on-demand Neo4j Cypher traversal for truly multi-hop queries.

- Personalization via per-user RATED history embedded as additional triplets and conditioned on session state.

- A learned cross-encoder re-ranker replacing the cosine re-rank.

- Expansion to larger graphs (IMDb, TMDB) and multi-modal upgrades incorporating posters and trailers.

10. References

[1] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," arXiv:2005.11401, 2020. https://arxiv.org/abs/2005.11401

[2] A. Hogan et al., "Knowledge Graphs," ACM Computing Surveys, vol. 54, no. 4, pp. 1-37, 2022. https://doi.org/10.1145/3447772

[3] "KG-Retriever: Efficient Knowledge Indexing for Retrieval-Augmented Large Language Models," arXiv:2412.05547, 2023. https://arxiv.org/html/2412.05547v1

[4] L. S. Nair and J. Cheriyan, "Multi-Featured Movie Recommendation Using Knowledge Graph," IDCIoT 2023. https://doi.org/10.1109/idciot56793.2023.10053435

[5] Z. Xu et al., "Retrieval-Augmented Generation with Knowledge Graphs for Customer Service Question Answering," SIGIR 2024. https://doi.org/10.1145/3626772.3661370

[6] Neo4j, "Example datasets - Getting Started," https://neo4j.com/docs/getting-started/appendix/example-data/

[7] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," EMNLP 2019. https://arxiv.org/abs/1908.10084

[8] J. Johnson, M. Douze, H. Jegou, "Billion-scale similarity search with GPUs," IEEE Trans. Big Data, 2019 (FAISS). https://arxiv.org/abs/1702.08734

[9] LangChain, "RetrievalQA Documentation," https://python.langchain.com/docs/modules/chains/popular/vector_db_qa

[10] Groq, "LLM Inference API Documentation," https://console.groq.com/docs